

المحاضرة 7

الدوال (التوايع): **functions**
مقدمة عن التوايع في لغة البرمجة **c++**
د. سهيل الحمود
د. أسامه ناصر

2025-2026

مقدمة عن الدوال Functions

- الدالة **Function** هي مجموعة تعليمات برمجية لها اسم محدد.
- يتم استدعاؤها لتنفيذ مهمة معينة داخل البرنامج.
- الهدف من استخدام الدوال:
 - تنظيم الكود وتقسيمه.
 - إعادة استخدام الكود.
 - تسهيل الفهم والصيانة.
 - تقليل التكرار.

لماذا نستخدم الدوال؟

- تخيل برنامجًا كبيرًا بدون دوال يحوي مئات المتغيرات العامة و الاجزاء المكررة
- باستخدام الدوال نستطيع:
 - تقسيم البرنامج إلى أجزاء منطقية.
 - اختبار كل جزء بشكل منفصل.
 - كتابة كود نظيف وواضح.

الدالة main()

- كل برنامج ++C يحتوي على دالة رئيسية اسمها `main()`
- يبدأ تنفيذ البرنامج منها.
- `main()` هي أول دالة يتم استدعاؤها تلقائياً عند تشغيل البرنامج.

```
int main() {  
    return 0;  
}
```

الشكل العام للدالة

```
return_type function_name(parameters) {  
    // statements  
    return value;  
}
```

- **Return Type** : نوع القيمة المرجعة
- **Function Name** : اسم الدالة
- **Parameters** : المعاملات (اختيارية)
- **Body** : جسم الدالة

مثال على دالة بسيطة

```
int sum(int a, int b) {  
    return a + b;  
}
```

- اسم الدالة: `sum`
- نوع القيمة الراجعة: `int`
- المعاملات: `a` و `b`
- القيمة المرجعة: مجموع العددين

تعريف وتصريح الدالة

1 تصريح الدالة (Function Prototype)

- يخبر المترجم بوجود دالة قبل استخدامها.

```
int sum(int, int);
```

2 تعريف الدالة

تحقيقها و تنفيذها

```
int sum(int a, int b) {  
    return a + b;  
}
```

لماذا نستخدم Function Prototype؟

- عندما تكون الدالة معرفة ومكتوبة بعد `main()`
- يمنع أخطاء الترجمة
- يجعل الكود أوضح ومنظمًا

استدعاء الدالة (Calling a Function)

```
int result = sum(3, 5);
```

- يتم استدعاء الدالة باستخدام اسمها.
- نمرر لها القيم المطلوبة (Arguments).
- يمكن تخزين الناتج في متغير.

مثال کامل

```
#include <iostream>
using namespace std;

int sum(int, int);

int main() {
    int x = 2, y = 4;
    cout << sum(x, y);
    return 0;
}

int sum(int a, int b) {
    return a + b;
}
```

الدوال التي ترجع قيمة

- تعيد قيمة باستخدام `return`
- يجب أن يطابق النوع المعروف في ترويسة التابع نوع القيمة التي تم إرجاعها

```
float average(int a, int b) {  
    return (a + b) / 2.0;  
}
```

الدوال بدون قيمة مرجعة (void)

- لا تعيد أي قيمة
- يمكن أن يكون لها آثار جانبية

```
void display() {  
    cout << "Hello World";  
}
```

المعاملات Parameters

- هي متغيرات شكلية (formal) تستقبل القيم داخل الدالة.
- تسمح بجعل الدالة عامة وقابلة لإعادة الاستخدام.

```
int square(int x) {  
    return x * x;  
}
```

Parameters vs Arguments

Arguments

Parameters

عند استدعاء الدالة

في تعريف الدالة

قيم فعلية

متغيرات

الاستدعاء بالقيمة (Call by Value)

- يتم إرسال نسخة من المتغير.
- أي تعديل لا يؤثر على المتغير الأصلي.

```
void change(int x) {  
    x = 10;  
}
```

الاستدعاء بالمرجع (Call by Reference)

- يتم إرسال عنوان المتغير.
- التعديل يؤثر على المتغير الأصلي.

```
void change(int &x) {  
    x = 10;  
}
```

مثال: تبدیل قیمتی متغیرین

```
void swap(int &a, int &b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

المعاملات الافتراضية (Default Arguments)

- يمكن إسناد قيم افتراضية إلى المعاملات بحيث يتم استخدامها في حال تم استدعاء التابع بدون إعطاء قيمة لهذه المعاملات.
- المعاملات الافتراضية يجب أن تكون آخر معاملات ضمن تعريف التابع

```
int add(int a, int b = 5) {  
    return a + b;  
}
```

التحميل الزائد للدوال (Function Overloading)

- يتم عندما يكون لدينا عدة دوال بنفس الاسم.
- هذه الدوال تختلف فيما بينها في عدد أو في نوع المعاملات.

```
int add(int a, int b);  
double add(double a, double b);
```

الدوال المضمنة Inline Functions

- تطلب من المترجم استبدال الدالة بالكود مباشرة.
- تستخدم للدوال الصغيرة.

```
inline int cube(int x) {  
    return x * x * x;  
}
```

نطاق المتغيرات (Scope)

- **Local Variable**: داخل الدالة فقط
- **Global Variable**: خارج جميع الدوال

```
int x = 5; // Global
```

أخطاء شائعة عند استخدام الدوال

- نسيان `return`
- اختلاف نوع القيمة الراجعة
- عدد معاملات خاطئ
- تحقيق بدون تصريح

الاستدعاء العودي Recursion؟

- الاستدعاء العودي هو أسلوب برمجي تقوم فيه الدالة **باستدعاء نفسها**.
- يُستخدم لحل المسائل التي يمكن تقسيمها إلى مسائل أصغر من نفس النوع.
- كل دالة عودية تتكوّن من جزأين أساسيين:
 - **الحالة الأساسية (Base Case)**
 - **الحالة العودية (Recursive Case)**

لماذا نستخدم Recursion؟

- تبسيط حل بعض المسائل المعقدة.
- مناسب لمسائل مثل:
 - حساب العامل $n!$ (Factorial)
 - متسلسلة فيبوناتشي
 - خوارزميات البحث في البنى الشجرية (لاحقا)
- يجعل الكود أقرب للوصف الرياضي للمشكلة.

الشكل العام للدالة العودية

```
return_type function_name(parameters) {  
    if (base_condition) {  
        return base_value;  
    }  
    return function(smaller_problem);  
}
```

مكونات الدالة العودية

```
int func(int n) {  
    if (n == 0)  
        return 1;  
    return n * func(n - 1);  
}
```

1 الحالة الأساسية Base Case

- توقف الاستدعاء العودي.
- بدونها يحدث استدعاء لا نهائي.

2 الحالة العودية Recursive Case

- تستدعي الدالة نفسها بقيم أصغر.

مثال 1: حساب مضروب العددي ! Factorial

الكود باستخدام Recursion

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

التعريف الرياضي

$$n! = n \times (n-1)!$$
$$0! = 1$$

Factorial(4)

تمثيل شجري للاستدعاء العودي

```
factorial(4)
├ factorial(3)
│   └ factorial(2)
│       └ factorial(1)
│           └ factorial(0)
```

خطوات تنفيذ

- 1. `factorial(4) → 4 * factorial(3)`
- 2. `factorial(3) → 3 * factorial(2)`
- 3. `factorial(2) → 2 * factorial(1)`
- 4. `factorial(1) → 1 * factorial(0)`
- 5.  `factorial(0) → 1`

مثال 2: متسلسلة فيبوناتشي Fibonacci

التعريف

هي سلسلة أعداد, كل عدد فيها يساوي مجموع العددين السابقين له. تبدأ السلسلة عادة بالرقمين 0, 1 وتكون على الشكل: 0, 1, 1, 2, 3, 5, 8, ...

$$\begin{aligned} F(n) &= F(n-1) + F(n-2) \\ F(0) &= 0, \quad F(1) = 1 \end{aligned}$$

كتابة كود لحساب فيوناتشي

ملاحظات

```
long fib(long n) {  
    if (n ≤ 1)  
        return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

- هذا الحل غير فعال للأرقام الكبيرة.
- يسبب عددًا كبيرًا من الاستدعاءات.
- الهدف منه تعليمي فقط لفهم الاستدعاء العودي Recursion.

Recursion vs Loop

Loop الحلقات

Recursion الاستدعاء العودي

أسرع في الأداء

أسهل للفهم في بعض المسائل

أقل استهلاكاً للذاكرة

يستهلك ذاكرة أكثر

أبسط في التنفيذ

يعتمد على بنية المكدس Stack

أخطاء شائعة في كتابة دالة عودية Recursion

- نسيان الحالة الأساسية
- الحالة الأساسية غير صحيحة
- عدم تقليل حجم المشكلة

استخدام الحلقات Loops بدل الاستدعاء العودي Recursion

- عند وجود حل تكراري بسيط
- عند الحاجة لسرعة تنفيذ و كفاءة باستخدام الذاكرة

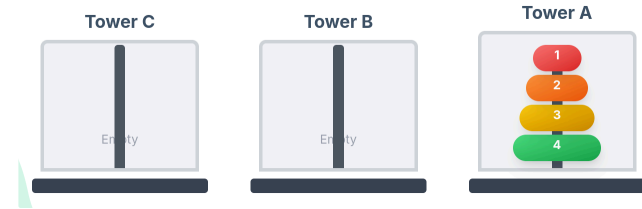
مثال آخر عن كتابة تابع عودي

طباعة الأرقام من n إلى 1 باستخدام Recursion

```
void print(int n) {  
    if (n == 0)  
        return;  
    cout << n << " ";  
    print(n - 1);  
}
```

مسألة أبراج هانوي (Tower of Hanoi)

- هي مسألة تقليدية تُستخدم لشرح مفهوم الاستدعاء العودي (Recursion).
- الهدف: نقل جميع الأقراص من البرج A إلى البرج C باستخدام البرج B كمساعد.
- القواعد:
 - نقل قرص واحد فقط في كل مرة.
 - يسمح بنقل القرص العلوي فقط من أي برج.
 - لا يجوز وضع قرص أكبر فوق قرص أصغر.
- أقل عدد حركات لحل المسألة هو: $2^n - 1$ حيث n عدد الأقراص.



Reset

Number of Disks

4

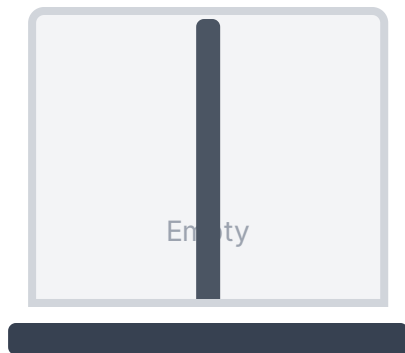
Minimum Moves

15

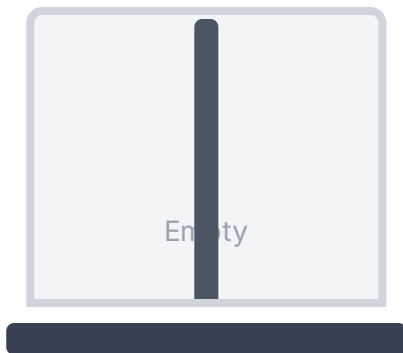
Moves

0

Tower C



Tower B



Tower A



```
#include <iostream>
using namespace std;

void hanoi(int n, char src, char dist, char aux) {
    if (n == 1) { // Base case
        cout << "Move disk 1 from "
              << src << " to " << dist << endl;
        return;
    }

    hanoi(n - 1, src, aux, dist);

    cout << "Move disk " << n << " from "
          << src << " to " << dist << endl;

    // Move n-1 disks from auxiliary to destination
    hanoi(n - 1, aux, dist, src);
}
```

أبراج هانوي Hanoi's Tower

```
int main() {  
    int n;  
    cout << "Enter number of disks: ";  
    cin >> n;  
  
    hanoi(n, 'A', 'C', 'B');  
    return 0;  
}
```